emphasis: Evolutionary Modeling for Phylogenetic Inference

Technical Reference Manual

Francisco Richter* Thijs Janzen Hanno Hildenbrandt

March 6, 2026

Contents

Abstract

emphasis is an R package for studying species diversification and its ecological drivers. It implements a general birth–death model in which per-lineage speciation and extinction rates depend on arbitrary state covariates: current species richness, total phylogenetic diversity, and lineage-specific evolutionary time. Because the marginal likelihood over the unobserved extinct lineages is analytically intractable, the package uses *importance sampling on augmented trees*: a proposal distribution augments the observed extant tree with stochastically drawn extinct lineages, and the resulting Monte Carlo estimator of the marginal log-likelihood serves as the objective for two complementary optimisers – a Monte Carlo Cross-Entropy Method with simulated annealing (CEM-SA) for global search and a Monte Carlo EM (MCEM) algorithm for local refinement. Model selection uses the Akaike Information Criterion computed from the IS log-likelihood at the optimum. All heavy computations are parallelised at the particle level using shared-memory multithreading via RcppParallel. This document provides a self-contained mathematical derivation of every component.

1 Notation

Table **??** summarises the principal symbols used throughout.

*University of Southern Switzerland. Email: `richtf@usi.ch`

2 The Diversification Model

2.1 General rate specification

The emphasis model belongs to the family of state-dependent birth–death processes (**??**). Every lineage s alive at time t (measured forward from the crown) is subject to a per-lineage speciation rate and extinction rate that depend on the current macroecological state of the clade:

$$\begin{aligned}\lambda(s,t;\boldsymbol{\theta}) &= f(\beta_0 + \beta_N N(t) + \beta_P P(t) + \beta_E E(s,t))\,,\\ \mu(s,t;\boldsymbol{\theta}) &= f(\gamma_0 + \gamma_N N(t) + \gamma_P P(t) + \gamma_E E(s,t))\,,\end{aligned} \tag{1}$$

where the full parameter vector is $\boldsymbol{\theta} = (\beta_0, \beta_N, \beta_P, \beta_E, \gamma_0, \gamma_N, \gamma_P, \gamma_E)^\top \in \mathbb{R}^8$.

State covariates.

$N(t)$ *Species richness.* The number of lineages currently alive. Negative β_N produces diversity-dependent suppression of speciation (ecological carrying capacity), while positive β_N encodes positive diversity feedbacks.

$P(t)$ *Phylogenetic diversity (PD).* The sum of all pendant branch-lengths: $P(t) = \sum_{s:\,\text{alive at } t} E(s,t)$. PD measures the ecological disparity available to the clade.

$E(s,t)$ *Evolutionary pendant time.* The time elapsed since lineage s last diverged. Long E may indicate accumulated reproductive isolation (higher λ) or competitive displacement (higher μ).

Link function. The link function $f : \mathbb{R} \to [0,\infty)$ ensures non-negative rates:

$$f(\eta) = \begin{cases} \max(0,\eta) & \text{linear link (default)},\\ \exp(\eta) & \text{exponential link.}\end{cases} \tag{2}$$

The linear link allows rates to reach exactly zero and is piecewise-linear in $\boldsymbol{\theta}$; the exponential link guarantees strictly positive rates. The exponential link is not supported for the EP covariate because no closed-form integral exists for $\int \exp(\gamma_0 + \gamma_E(t - a_s))\,\mathrm{d}t$ combined with other covariates.

2.2 Model shortcuts

Setting subsets of $\{\beta_N, \beta_P, \beta_E, \gamma_N, \gamma_P, \gamma_E\}$ to zero yields the four canonical models (Table **??**).

The number of free parameters is always $p = 2 + 2k$, where $k \in \{0,1,2,3\}$ is the number of active covariates.

3 Stochastic Simulation of Phylogenies

3.1 Forward simulation

Forward simulation of the state-dependent birth–death process proceeds via Gillespie's direct method (**?**). At each step the algorithm:

1. Computes the total event rate $R(t) = \sum_{s:\,\text{alive}}[\lambda(s,t;\boldsymbol{\theta}) + \mu(s,t;\boldsymbol{\theta})]$.
2. Draws the waiting time to the next event, $\Delta t \sim \text{Exp}(R(t))$.
3. Selects which lineage and event type (speciation or extinction) with probability proportional to individual rates.
4. Updates the state $(N, P, \{E_s\})$ and the lineage set.

Because $N(t)$, $P(t)$, and each $E(s,t)$ change only at events, $R(t)$ is piecewise-constant between events and the Gillespie step is exact. For the EP covariate, where $E(s,t)$ grows continuously, the waiting time no longer has a closed-form distribution and is drawn instead via *thinning* (Lewis–Shedler algorithm (**?**)): candidate times are drawn from a homogeneous Poisson process at rate $\lambda_{\max} \geq \sup_t R(t)$, and each candidate is accepted with probability $R(t^*)/\lambda_{\max}$.

3.2 Conditional simulation (augmentation)

Given the observed extant tree $\mathcal{T}$, the package draws the unobserved extinct lineages z from the proposal distribution $q(z \mid \mathcal{T}, \boldsymbol{\theta})$, which conditions on $\mathcal{T}$ being the extant subtree. The algorithm operates interval by interval between consecutive branching times $(t_1 > t_2 > \cdots > t_n)$ of $\mathcal{T}$:

1. Within each interval $[t_{i+1}, t_i]$, candidate extinction events are generated by thinning a Poisson process at rate $\lambda_{\max}$.
2. Each candidate time t^* is accepted as an extinction event with probability $\lambda(t^*;\boldsymbol{\theta})/\lambda_{\max}$.
3. An accepted extinction at t^* spawns a new lineage, which is then forward-simulated from t^* until it goes extinct before $t = 0$ (the present); if it does not go extinct it is discarded and redrawn.
4. The procedure recurses on the extinct branches.

The log-proposal probability

$$\ell_i^{(q)} = \log q(z_i \mid \mathcal{T}, \boldsymbol{\theta}) \tag{3}$$

is accumulated analytically during each simulation pass and returned alongside z_i and $\ell_i^{(f)}$, so that importance-sampling reweighting can be applied immediately without re-running any simulation.

3.3 Batch simulation and survival probability

Passing a parameter matrix to `simulate_tree()` triggers batch mode: each row $\boldsymbol{\theta}_r$ is simulated independently, with failed runs (clade extinction or size overflow) retained. The fraction of simulations that produce a surviving clade estimates the probability of non-extinction:

$$\hat{\pi}(\boldsymbol{\theta}) = \frac{1}{R}\sum_{r=1}^{R} \mathbf{1}[\text{simulation } r \text{ survived}]. \tag{4}$$

This quantity can be used to correct the likelihood for conditioning on non-extinction. For diversity-dependent models, `train_GAM()` fits a binomial generalised additive model to the outcomes of a batch run, producing a smooth estimate of $\pi(\boldsymbol{\theta})$ over the parameter space that can be evaluated at any $\boldsymbol{\theta}$ without further simulation.

3.4 Parallel simulation

The C++ simulation kernel is parallelised using RcppParallel, which provides a portable thread pool backed by Intel TBB on supported platforms and a fallback `std::thread` implementation elsewhere. Each particle (i.e. each augmented tree draw) is independent: particle i runs its own thinning loop, accumulates its own $(\ell_i^{(f)}, \ell_i^{(q)})$ pair, and writes to a pre-allocated per-particle slot with no shared mutable state. The thread pool divides the S particles across `num_threads` workers. Concretely, the C++ function `augment_trees()` accepts an integer `num_threads` argument; setting `num_threads = 1` (default) runs everything sequentially, while any value > 1 activates the parallel worker pool. The number of threads should not exceed the number of physical cores, and diminishing returns are expected when the per-particle compute time is small relative to thread-scheduling overhead.

4 The Phylogenetic Likelihood

4.1 Observed versus augmented data

The observed data $\mathcal{T}$ consists of the extant tree: branching times $\mathbf{t}$ and tip count n. The unobserved complete data is the augmented tree $\mathcal{T}^+ = (\mathcal{T}, z)$, where z denotes all extinct lineages. The marginal likelihood integrates over all possible extinct histories:

$$p(\mathcal{T} \mid \boldsymbol{\theta}) = \int p(\mathcal{T}, z \mid \boldsymbol{\theta})\, \mathrm{d}\nu(z), \tag{5}$$

where ν is the natural dominating measure on augmented-tree space (counting measure over the combinatorial structure times Lebesgue measure over the continuous event times). This integral is analytically tractable for the CR and DD models (**?**) but is intractable for EP and any model with lineage-specific covariates.

4.2 Complete-data log-likelihood

Let the augmented tree $\mathcal{T}^+$ have K total lineages, lineage k alive on the time interval $[a_k, d_k]$ where a_k is its birth time and d_k its death (or the crown age for extant lineages). The complete-data log-likelihood of the birth–death process is:

$$\log p(\mathcal{T}, z \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} \log r_k(t_k^{\mathrm{ev}}) - \sum_{k=1}^{K} \int_{a_k}^{d_k} \Big[\lambda(k, t; \boldsymbol{\theta}) + \mu(k, t; \boldsymbol{\theta})\Big] \, \mathrm{d}t, \tag{6}$$

where $r_k(t_k^{\mathrm{ev}})$ is the rate of the event terminating lineage k (λ for speciation, μ for extinction), and the integral is the cumulative hazard of lineage k over its lifetime. Expanding:

$$\begin{aligned} \log p(\mathcal{T}, z \mid \boldsymbol{\theta}) = & \sum_{k:\,\text{speciation}} \log \lambda(k, t_k^{\mathrm{ev}}; \boldsymbol{\theta}) + \sum_{k:\,\text{extinction}} \log \mu(k, t_k^{\mathrm{ev}}; \boldsymbol{\theta}) \\ & - \sum_{k=1}^{K} \int_{a_k}^{d_k} \Big[\lambda(k, t; \boldsymbol{\theta}) + \mu(k, t; \boldsymbol{\theta})\Big] \, \mathrm{d}t. \end{aligned} \tag{7}$$

Closed-form hazard integrals. For the linear link, $\lambda(k, t; \boldsymbol{\theta}) = \max(0, \eta_k(t))$ and the integral $\int_{a_k}^{d_k} \eta_k(t) \, \mathrm{d}t$ has a closed form for all supported covariate combinations, because $N(t)$, $P(t)$, and $E(k, t)$ are all piecewise-linear between events. The C++ kernel precomputes the integral for each lineage interval analytically, making evaluation of (**??**) exact and fast.

5 Importance-Sampling Likelihood Estimation

5.1 The IS estimator

Let $z_1, \ldots, z_S \overset{\text{iid}}{\sim} q(z \mid \mathcal{T}, \boldsymbol{\theta})$. The marginal likelihood (**??**) is the expectation

$$p(\mathcal{T} \mid \boldsymbol{\theta}) = \mathbb{E}_q \left[\frac{p(\mathcal{T}, z \mid \boldsymbol{\theta})}{q(z \mid \mathcal{T}, \boldsymbol{\theta})} \right], \tag{8}$$

estimated without bias by

$$\widehat{L}_S(\boldsymbol{\theta}) = \frac{1}{S} \sum_{i=1}^{S} \exp\Big(\ell_i^{(f)} - \ell_i^{(q)}\Big) = \frac{1}{S} \sum_{i=1}^{S} \exp(w_i). \tag{9}$$

The IS log-likelihood estimate is therefore

$$\widehat{f}(\boldsymbol{\theta}) = \log \widehat{L}_S(\boldsymbol{\theta}) = \log \left(\frac{1}{S} \sum_{i=1}^{S} \exp(w_i) \right), \tag{10}$$

evaluated in numerically stable form via the log-sum-exp identity:

$$\widehat{f}(\boldsymbol{\theta}) = w_{\max} + \log\left(\frac{1}{S}\sum_{i=1}^{S}\exp(w_i - w_{\max})\right), \quad w_{\max} = \max_i w_i. \tag{11}$$

Remark 1 (Jensen's inequality bias)**.** *Because* log *is concave,* $\widehat{f}(\boldsymbol{\theta}) \le \mathbb{E}_q[w]$ *by Jensen's inequality, so* $\widehat{f}$ *underestimates* $\log p(\mathcal{T} \mid \boldsymbol{\theta})$*. The bias is* $O(1/S)$ *and vanishes as* $S \to \infty$*.*

5.2 Bias-corrected estimator

A moment-based correction removes the leading-order Jensen bias. Define $\ell_{\text{hat}} = S^{-1}\sum_i w_i$ and the central moments $m_k = S^{-1}\sum_i (w_i - \ell_{\text{hat}})^k$. The exact identity relating $\log \mathbb{E}[e^W]$ to the cumulants of W gives:

$$\widehat{f}_{\text{BC}}(\boldsymbol{\theta}) = \ell_{\text{hat}} + \log\left(1 + \sum_{k=2}^{K}\frac{m_k}{k!}\right). \tag{12}$$

At $K = 2$ this is the log-normal correction $\ell_{\text{hat}} + \log(1 + m_2/2) \approx \ell_{\text{hat}} + m_2/2$ for small m_2. The full series ($K \to \infty$) recovers $\widehat{f}$ exactly; the truncation at finite K is the approximation.

Handling rejected trees. Trees that exceed $\lambda_{\max}$ or the maximum extinct-lineage count have $L_i = 0$ ($w_i = -\infty$). They are excluded from moment sums but enter the denominator:

$$\widehat{f}(\boldsymbol{\theta}) = \widehat{f}_{\text{valid}}(\boldsymbol{\theta}) + \log\left(\frac{S_{\text{valid}}}{S}\right), \tag{13}$$

where $\widehat{f}_{\text{valid}}$ uses only the S_{valid} non-rejected trees.

5.3 Effective sample size

The IS effective sample size quantifies IS quality:

$$\text{ESS} = \frac{\left(\sum_{i=1}^{S} e^{w_i}\right)^2}{\sum_{i=1}^{S} e^{2w_i}}. \tag{14}$$

ESS $\approx S$ means roughly uniform weights (efficient IS); ESS ≈ 1 means one tree dominates (IS collapse). We report ESS$/S \in (0, 1]$ as a diagnostic. For reliable inference, ESS$/S \ge 0.1$ is recommended.

5.4 Cross-importance sampling (Mode 2)

In Mode 2 (`shared_trees = TRUE`), a single pool of S augmented trees is drawn at a reference parameter $\boldsymbol{\theta}_0$, and $\widehat{f}$ is evaluated at any $\boldsymbol{\theta}_j$ by re-scoring the same trees:

$$\widehat{f}(\boldsymbol{\theta}_j) = \log\left(\frac{1}{S}\sum_{i=1}^{S}\exp\Big(\ell^{(f)}(z_i, \boldsymbol{\theta}_j) - \ell_i^{(q)}\Big)\right), \tag{15}$$

where $\ell_i^{(q)} = \log q(z_i \mid \mathcal{T}, \boldsymbol{\theta}_0)$ are fixed at simulation time and $\ell^{(f)}(z_i, \boldsymbol{\theta}_j)$ is re-evaluated analytically at $\boldsymbol{\theta}_j$ via `eval_logf()`. This reduces the total augmentations from $M \times S$ to S per CEM iteration, at the cost of higher variance when $\boldsymbol{\theta}_j$ and $\boldsymbol{\theta}_0$ are distant. Mode 2 is parallelised over the M particles (scoring is cheap), while Mode 1 parallelises the S augmentation draws.

6 The Cross-Entropy Method with Simulated Annealing

6.1 The optimisation problem

Parameter estimation reduces to maximising the IS log-likelihood:

$$\hat{\boldsymbol{\theta}} = \operatorname*{arg\,max}_{\boldsymbol{\theta}\in[\boldsymbol{\ell},\,\mathbf{u}]} \widehat{f}(\boldsymbol{\theta}). \tag{16}$$

Because $\widehat{f}$ is stochastic (it depends on randomly drawn augmented trees) and potentially multimodal, gradient-based methods are unreliable. The Cross-Entropy Method (CEM) (**??**) solves (**??**) by maintaining an evolving population of candidate parameter vectors and adaptively concentrating it near the optimum.

6.2 Cross-entropy population update

Let $g(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ be a parametric family of densities over the search box $[\boldsymbol{\ell}, \mathbf{u}]$. At each iteration the CEM selects the member of g that minimises the KL divergence to the "ideal" distribution g^* that concentrates all mass at the global maximiser of $\widehat{f}$:

$$(\boldsymbol{\mu}^{(k+1)}, \boldsymbol{\Sigma}^{(k+1)}) =_{\boldsymbol{\mu},\boldsymbol{\Sigma}} \mathrm{KL}\Big(g^* \,\Big\|\, g(\cdot; \boldsymbol{\mu}, \boldsymbol{\Sigma})\Big). \tag{17}$$

In the Gaussian family this CE update reduces to computing the sample mean and covariance of the elite particles. Because the full covariance update can overfit when the elite set is small, emphasis uses a diagonal approximation: each parameter dimension is updated independently, giving a product of independent Gaussian perturbations.

Each CEM iteration consists of three steps — **Evaluation**, **Selection**, and **Update** — described in the following subsections and collected in Algorithm **??**. Note that these are *not* the E-step and M-step of the EM algorithm; the CEM has no latent-variable expectation and no Q-function maximisation.

6.3 Simulated annealing schedule

To prevent premature convergence, the perturbation variance is not derived purely from the elite sample but is controlled by an explicit *annealing schedule* analogous to simulated annealing (**?**). Concretely, define the "temperature" vector $\boldsymbol{\sigma}^{(k)} \in \mathbb{R}^8_{>0}$ as the per-parameter perturbation standard deviation at iteration k. New particles are generated as:

$$\boldsymbol{\theta}_j^{(k+1)} = \mathrm{clip}\Big(\boldsymbol{\theta}_{\pi(j)}^{(k)} + \boldsymbol{\varepsilon}_j,\ \boldsymbol{\ell},\ \mathbf{u}\Big), \quad \boldsymbol{\varepsilon}_j \sim \mathcal{N}\Big(\mathbf{0},\ \mathrm{diag}\Big(\boldsymbol{\sigma}^{(k)}\Big)^2\Big), \tag{18}$$

where $\pi(j)$ selects uniformly from the elite set and $\mathrm{clip}(\cdot, \boldsymbol{\ell}, \mathbf{u})$ projects back into the feasible box.

The temperature is cooled linearly:

$$\boldsymbol{\sigma}^{(k+1)} = \boldsymbol{\sigma}^{(k)} - \alpha\,\mathbf{1}, \qquad \alpha = \frac{\boldsymbol{\sigma}^{(0)}}{T}, \tag{19}$$

with the initial temperature set to one quarter of the search range:

$$\sigma_p^{(0)} = \frac{u_p - \ell_p}{4}, \quad p = 1, \ldots, 8. \tag{20}$$

With this schedule, $\boldsymbol{\sigma}^{(T)} = \mathbf{0}$: the population fully collapses at iteration T. The linear decay is analogous to the linear cooling schedule in classical simulated annealing and guarantees a smooth transition from broad exploration (large σ, wide Gaussian perturbations) to fine-grained exploitation (small σ, narrow perturbations near the current elite). Early stopping via the plateau and annealing rules (Section **??**) allows termination before T is reached.

Connection to simulated annealing. In classical simulated annealing, the Boltzmann acceptance probability at temperature τ is $\exp(-\Delta E/\tau)$: high τ accepts bad moves freely (exploration); low τ accepts only improvements (exploitation). In the CEM-SA framework, $\boldsymbol{\sigma}^{(k)}$ plays the role of τ: large σ generates particles far from the current elite (exploration); as $\sigma \to 0$ only particles close to elite copies are generated (exploitation). The crucial difference is that CEM-SA evaluates the stochastic objective $\widehat{f}$ directly at many candidate points in parallel rather than performing a single-chain random walk.

6.4 Stopping rules

Three criteria are checked in order at the end of each iteration:

1. **Annealing exhausted.** $\boldsymbol{\sigma}^{(k)} \leq \mathbf{0}$ element-wise: the population has fully collapsed; further perturbation would be zero.

2. **Plateau.** The improvement $f^{(k)} - f^{(k-1)} < \delta$ (default $\delta = 10^{-4}$) for P (default 5) consecutive iterations. This guards against wasting computation when the IS surface is flat near the optimum.

3. **Hard cap.** $k = T$ (default $T = 20$).

6.5 Algorithm

The complete CEM-SA procedure is given in Algorithm **??**.

Algorithm 1: Monte Carlo CEM with Simulated Annealing (`emphasis_cem`)

Input: branching times $\mathbf{t}$; bounds $[\boldsymbol{\ell}, \mathbf{u}]$; population size M; elite fraction ρ; IS sample size S; max iterations T; annealing step α; plateau tolerance δ; patience P; `num_threads`

Output: best parameter estimate $\hat{\boldsymbol{\theta}}$; per-iteration history

$\boldsymbol{\sigma}^{(0)} \leftarrow (u_p - \ell_p)/4$ for each p `// initial temperature` (**??**)
Initialise $\{\boldsymbol{\theta}_j^{(0)}\}_{j=1}^{M}$ uniformly in $[\boldsymbol{\ell}, \mathbf{u}]$
$k \leftarrow 0$; $\quad plat \leftarrow 0$
while $k < T$ **do**
  if $\boldsymbol{\sigma}^{(k)} \leq \mathbf{0}$ **then stop** (*annealing exhausted*)
  `// Step 1 - Evaluation: score every particle (in parallel)`
  for $j = 1, \ldots, M$ **do** `// num_threads workers`
    Draw $z_1^{(j)}, \ldots, z_S^{(j)} \sim q(\cdot \mid \mathcal{T}, \boldsymbol{\theta}_j^{(k)})$
    Compute $\widehat{f}(\boldsymbol{\theta}_j^{(k)})$ via (**??**); mark invalid if result is NA
  `// Step 2 - Selection: identify elite particles`
  $j^* \leftarrow \arg\max_{j:\,\text{valid}} \widehat{f}(\boldsymbol{\theta}_j^{(k)})$; $\quad f^{(k)} \leftarrow \widehat{f}(\boldsymbol{\theta}_{j^*}^{(k)})$
  Elite $\leftarrow$ top $\lfloor \rho M \rfloor$ valid particles ranked by $\widehat{f}$
  `// Plateau stopping check`
  if $k > 0$ ***and*** $f^{(k)} - f^{(k-1)} < \delta$ **then**
    $plat \leftarrow plat + 1$
    if $plat \geq P$ **then stop** (*plateau*)
  else
    $plat \leftarrow 0$
  `// Step 3 - Update: resample around elite particles and cool`
  For each non-elite slot j: draw $\pi(j) \sim \text{Uniform(Elite)}$ and generate $\boldsymbol{\theta}_j^{(k+1)}$ via (**??**)
  Copy elite particles unchanged: $\boldsymbol{\theta}_j^{(k+1)} \leftarrow \boldsymbol{\theta}_j^{(k)}$ for $j \in$ Elite
  $\boldsymbol{\sigma}^{(k+1)} \leftarrow \boldsymbol{\sigma}^{(k)} - \alpha\,\mathbf{1}$
  $k \leftarrow k + 1$
$\hat{\boldsymbol{\theta}} \leftarrow \arg\max_k f^{(k)}$ `// global best across all iterations`

Parallelism in the evaluation step. The inner loop over particles $j = 1, \ldots, M$ is the dominant cost. Each augmentation is entirely independent: particle j accesses only its own parameter vector $\boldsymbol{\theta}_j^{(k)}$ and writes to its own pre-allocated result slot. This embarrassing parallelism is exploited by RcppParallel: the M particles are partitioned into `num_threads` contiguous blocks, each processed by a separate thread from the TBB worker pool. Because

there is no inter-particle communication and each thread writes to a disjoint memory region, no locks or atomic operations are needed, and parallel efficiency is near-linear up to the number of physical cores.

Particle rescue. If every particle in a given iteration produces an invalid $\widehat{f}$ (all augmented trees rejected), the per-particle limits are escalated: $\lambda_{\max} \leftarrow 10\lambda_{\max}$, $N_{\max} \leftarrow 10N_{\max}$, and the E-step is repeated. If $N_{\max} > 10{,}000$ with all particles still invalid, the run terminates with a warning advising the user to increase `num_points` or widen the bounds.

7 Monte Carlo EM for Local Refinement

7.1 The EM objective

The Monte Carlo Expectation–Maximisation (MCEM) algorithm (**??**) maximises the expected complete-data log-likelihood:

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{q(\cdot \mid \mathcal{T}, \boldsymbol{\theta}^{(t)})}\Big[\log p(\mathcal{T}, z \mid \boldsymbol{\theta})\Big]. \tag{21}$$

7.2 IS-weighted E-step

At iteration t, draw $z_1, \ldots, z_S \sim q(\cdot \mid \mathcal{T}, \boldsymbol{\theta}^{(t)})$. The IS-weighted approximation to Q is:

$$\hat{Q}(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \frac{\sum_{i=1}^{S} \tilde{w}_i^{(t)} \cdot \ell^{(f)}(z_i, \boldsymbol{\theta})}{1}, \quad \tilde{w}_i^{(t)} = \frac{e^{w_i^{(t)}}}{\sum_j e^{w_j^{(t)}}}, \tag{22}$$

where $w_i^{(t)} = \ell^{(f)}(z_i, \boldsymbol{\theta}^{(t)}) - \ell_i^{(q)(t)}$ and $\tilde{w}_i^{(t)}$ are normalised IS weights. This is the self-normalised IS estimator of Q, which is consistent for any positive proposal $q > 0$ wherever $p > 0$.

7.3 M-step and convergence

The M-step maximises $\hat{Q}(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$ over $\boldsymbol{\theta} \in [\boldsymbol{\ell}, \mathbf{u}]$ using the derivative-free COBYLA algorithm (**?**) via the `nloptr` interface. COBYLA constructs successive linear approximations to the objective from function evaluations, which is appropriate here because the IS noise makes numerical gradients unreliable.

Convergence is declared when the change in $\widehat{f}(\boldsymbol{\theta}^{(t)})$ across consecutive post-burnin iterations falls below a tolerance ε (default 0.1). A burn-in phase (default 20 iterations) is excluded from the convergence check to allow transients to decay.

7.4 MCEM as a refinement stage

Because MCEM converges to a local maximum of $\boldsymbol{\theta} \mapsto \log p(\mathcal{T} \mid \boldsymbol{\theta})$, the recommended workflow is:

1. Run CEM-SA to locate a good region (global search).
2. Warm-start MCEM at $\hat{\boldsymbol{\theta}}_{\text{CEM}}$ (local refinement).

The MCEM E-step parallelises over the S augmented trees in the same way as the CEM E-step, using `num_threads` threads.

8 Model Selection

8.1 AIC

For fitted model $\mathcal{M}_k$ with p_k free parameters and IS log-likelihood $\widehat{f}(\hat{\boldsymbol{\theta}}_k)$:

$$\text{AIC}_k = -2\,\widehat{f}(\hat{\boldsymbol{\theta}}_k) + 2p_k. \tag{23}$$

The preferred model minimises AIC. The gaps $\Delta\text{AIC}_k = \text{AIC}_k - \min_j \text{AIC}_j \geq 0$ quantify evidence against model k: $\Delta\text{AIC} < 2$ is considered negligible, $2 \leq \Delta\text{AIC} < 6$ moderate, and $\Delta\text{AIC} \geq 6$ strong.

Remark 2 (IS bias in AIC comparisons)**.** *Because $\widehat{f}$ underestimates $\log p(\mathcal{T} \mid \hat{\boldsymbol{\theta}})$ (Remark **??**), the AIC is upward-biased. When models are evaluated at independently drawn augmented trees (Mode 1), the bias is model-specific and does not cancel in ΔAIC. This residual Monte Carlo noise is quantified by the bootstrap variance (Section **??**) and is reduced by increasing S or using Mode 2.*

8.2 Bootstrap variance of $\widehat{f}$

To quantify IS Monte Carlo uncertainty at the optimum, emphasis provides a leave-half-out subsampling bootstrap:

$$\widehat{\text{Var}}[\widehat{f}(\hat{\boldsymbol{\theta}})] = \frac{1}{B}\sum_{b=1}^{B}\Big(\widehat{f}^{(b)}(\hat{\boldsymbol{\theta}}) - \bar{\widehat{f}}(\hat{\boldsymbol{\theta}})\Big)^2, \tag{24}$$

where $\widehat{f}^{(b)}$ is computed on a uniformly drawn half of the S_{final} trees and $\bar{\widehat{f}}$ is the mean over B replicates. If $\widehat{\text{Var}}^{1/2}[\widehat{f}] \sim |\Delta\text{AIC}|/2$, the model comparison is noise-dominated and S should be increased.

8.3 Automated three-model selection

`select_diversification_model()` fits the three biologically most relevant models (DD, PD, EP) in a single call, applies (**??**), and returns a ranked comparison table with AIC weights $w_k = e^{-\Delta\mathrm{AIC}_k/2} / \sum_j e^{-\Delta\mathrm{AIC}_j/2}$.

9 Diagnostics

9.1 CEM convergence diagnostics

`diagnose_cem()` extracts and plots four diagnostic panels:

Log-likelihood trace. $f^{(k)}$ vs. iteration k, annotated with the stopping rule. A monotone increasing curve ending at a plateau indicates healthy convergence.

Parameter traces. $\hat{\boldsymbol{\theta}}_p^{(k)}$ vs. k for each dimension p. Stable late-iteration values confirm convergence.

Population spread. Median and IQR of $\widehat{f}(\boldsymbol{\theta}_j^{(k)})$ over all valid particles. Narrowing IQR reflects progressive collapse.

IS weight distribution. Histogram of w_i at the globally best particle, annotated with ESS. A concentrated unimodal histogram near the mean indicates healthy IS weights.

9.2 GAM-based survival probability

`train_GAM()` fits a binomial GAM to batch-simulation outcomes, estimating $\pi(\boldsymbol{\theta}) = \Pr(\text{clade survives} \mid \boldsymbol{\theta})$ as a smooth surface. `predict_survival()` evaluates this surface at any $\boldsymbol{\theta}$ without re-simulating, enabling fast likelihood corrections for conditioning on non-extinction.

10 Implementation

10.1 Software architecture

emphasis is written in R and C++17, with the computationally intensive steps delegated via Rcpp. The public API consists of five primary functions:

`simulate_tree()` Forward and conditional simulation; dispatches to the C++ Gillespie/thinning engine.

`augment_trees()` E-step primitive: draw S augmented trees, return `trees`, `logf`, `logg`.

`eval_logf()` Re-score cached trees at a new $\boldsymbol{\theta}$ (enables Mode 2 cross-IS, Section **??**).

`estimate_rates()` Unified inference entry point; dispatches to CEM-SA or MCEM and returns a named `emphasis_fit` object.

`compare_models()` / `select_diversification_model()`
AIC-based model comparison.

10.2 Separation of simulation and scoring

The key architectural principle is the clean separation of three concerns:

1. **Simulation.** `augment_trees()` simulates trees and computes $\ell^{(f)}$ and $\ell^{(q)}$ in a single pass.

2. **Scoring.** `eval_logf()` re-evaluates $\ell^{(f)}(z, \boldsymbol{\theta}')$ for a new $\boldsymbol{\theta}'$ on already-simulated trees, without re-running the stochastic augmentation.

3. **IS aggregation.** `.is_fhat(logf, logg)` in R combines $\ell^{(f)}$ and $\ell^{(q)}$ into $\widehat{f}$ via (**??**).

This design is what makes Mode 2 possible: one simulation pass at $\boldsymbol{\theta}_0$ produces trees reused across all M particles.

10.3 Parallel computing

All parallelism in emphasis is shared-memory multithreading implemented via RcppParallel, which provides:

- A *portable thread pool* backed by Intel TBB (`tbb::parallel_for`) on Linux, macOS, and Windows, with a fallback `std::thread` implementation when TBB is unavailable.

- *Thread-safe memory* via pre-allocated per-thread result vectors: each thread writes exclusively to its own slice of the output array with no shared mutable state, so no locks or atomics are needed.

- *Work scheduling*: the S (Mode 1) or M (Mode 2) tasks are divided into `num_threads` contiguous blocks. The block boundaries are fixed at launch time, so there is no dynamic work stealing (which keeps overhead minimal for uniform per-task cost).

Practical guidance.

- Set `num_threads` to the number of *physical* cores (not hyperthreads); hyperthreading rarely helps for floating-point intensive workloads.

- Parallel speedup is near-linear when $S \gg$ `num_threads` and per-tree compute time dominates thread-launch overhead. For very small S (e.g. $S = 1$, the CEM default), single-threaded mode is optimal.
- In Mode 1, parallelise the S trees per particle (`num_threads` > 1 inside `augment_trees`). In Mode 2, the M re-scorings in `eval_logf` are parallelised instead, since augmentation itself is already done.

10.4 Parameter layout

All C++ functions operate on the full 8-element parameter vector in the canonical order $(\beta_0, \beta_N, \beta_P, \beta_E, \gamma_0, \gamma_N, \gamma_P, \gamma_E)^\top$. Inactive parameters are set to zero. The model binary flag $\mathbf{m} = (m_N, m_P, m_E) \in \{0, 1\}^3$ gates which covariate terms are included in the augmentation proposal. For the EP covariate, $m_E = 1$ additionally switches the C++ rate functions from standard to EP-aware mode in `eval_logf`, since the EP contribution to the hazard integral (**??**) requires a different analytic formula.

11 Simulation Studies

11.1 Estimation power

To evaluate the frequentist accuracy of CEM estimation under the PD model, we simulate $n = 100$ independent trees from the true parameter vector

$$\boldsymbol{\theta}^* = (\beta_0, \beta_P, \gamma_0, \gamma_P) = (0.6,\ 0.005,\ 0.15,\ -0.003),$$

with crown age $T = 8$ and estimate $\boldsymbol{\theta}$ on each replicate via CEM with $M = 100$, $S = 1$, $T = 20$ iterations. For each replicate r we record the estimate $\hat{\boldsymbol{\theta}}_r$ and the number of extant tips n_r. Accuracy is summarised by:

$$\text{Bias}_p = \frac{1}{n} \sum_{r=1}^{n} (\hat{\boldsymbol{\theta}}_{r,p} - \boldsymbol{\theta}^*_p), \tag{25}$$

$$\text{RMSE}_p = \sqrt{\frac{1}{n} \sum_{r=1}^{n} (\hat{\boldsymbol{\theta}}_{r,p} - \boldsymbol{\theta}^*_p)^2}. \tag{26}$$

The tree-size effect is visualised by plotting $|\hat{\boldsymbol{\theta}}_{r,p} - \boldsymbol{\theta}^*_p|$ against n_r with a LOWESS smooth. Monotone decrease in error with n_r confirms consistency: larger trees produce a sharper IS likelihood surface.

11.2 Model selection power

We assess AIC-based model selection in a fully crossed design. Twenty-five trees are simulated from each of the four pure models (CR, DD, PD, EP), giving 100 trees total. All four models are fitted to each tree via CEM, and the model with the lowest AIC is selected. Results are summarised by the 4×4 confusion matrix

$$C_{ij} = \#\Big\{\text{trees from model } i \text{ for which model } j \text{ is selected}\Big\}, \tag{27}$$

overall accuracy $\mathrm{acc} = \mathrm{tr}(C)/\sum_{i,j} C_{ij}$, and per-model recall $C_{ii}/\sum_j C_{ij}$.

12 Summary of Control Parameters

13 Mathematical Appendix

13.1 Log-sum-exp identity

For numerical stability (**??**) is evaluated as:

$$\log\left(\frac{1}{S}\sum_{i=1}^{S} e^{w_i}\right) = \log\left(\frac{e^{w_{\max}}}{S}\sum_{i=1}^{S} e^{w_i - w_{\max}}\right) = w_{\max} + \log\left(\frac{1}{S}\sum_{i=1}^{S} e^{w_i - w_{\max}}\right). \tag{28}$$

Since $w_i - w_{\max} \leq 0$, all terms $e^{w_i - w_{\max}} \in (0, 1]$ are safe.

13.2 ESS in log-space

Define normalised weights $\tilde{w}_i = e^{w_i}/\sum_j e^{w_j}$. Then $\mathrm{ESS} = 1/\sum_i \tilde{w}_i^2$, which in log-space is:

$$\mathrm{ESS} = \exp\Big[2\,\mathrm{lse}(\mathbf{w}) - \mathrm{lse}(2\mathbf{w})\Big], \tag{29}$$

where $\mathrm{lse}(\mathbf{v}) = \max_i v_i + \log \sum_i e^{v_i - \max_j v_j}$.

13.3 Annealing decay rate

With $\boldsymbol{\sigma}^{(0)} = \boldsymbol{\sigma}_0$ and linear decay $\boldsymbol{\sigma}^{(k)} = \boldsymbol{\sigma}_0 - k\alpha\mathbf{1}$, the population collapses ($\sigma_p^{(k^*)} \leq 0$) at $k^* = \lceil \sigma_{0,p}/\alpha \rceil$. Setting $k^* = T$ for all p gives $\alpha = \sigma_{0,p}/T$ as in (**??**). This ensures the annealing-exhausted stopping rule *would* trigger exactly at iteration T in the absence of plateau stopping.

13.4 Hazard integral for the linear link

For lineage k alive on $[a_k, d_k]$ under the linear link, the hazard integral decomposes as:

$$H_k = \int_{a_k}^{d_k} \Big[f(\eta_\lambda(k,t)) + f(\eta_\mu(k,t)) \Big] \, \mathrm{d}t, \tag{30}$$

where $\eta_\lambda(k,t) = \beta_0 + \beta_N N(t) + \beta_P P(t) + \beta_E E(k,t)$ and similarly for η_μ. Since $N(t)$, $P(t)$, and $E(k,t)$ are all piecewise-linear in t between consecutive events, each sub-interval $[t_j, t_{j+1}]$ within $[a_k, d_k]$ has a constant N, P, and linearly increasing $E(k,t) = t - a_k$. On such a sub-interval with $f(\eta) = \max(0, \eta)$:

$$\int_{t_j}^{t_{j+1}} \max(0,\, c_0 + c_1 t) \, \mathrm{d}t = \begin{cases} c_0(t_{j+1} - t_j) + \frac{c_1}{2}(t_{j+1}^2 - t_j^2) & \text{if } c_0 + c_1 t \geq 0 \text{ on } [t_j, t_{j+1}], \\ 0 & \text{if } c_0 + c_1 t \leq 0 \text{ on } [t_j, t_{j+1}], \\ \text{partial integral} & \text{otherwise (split at root).} \end{cases} \tag{31}$$

The C++ kernel evaluates this analytically for each sub-interval, making the complete-data log-likelihood (**??**) exact.

References

de Boer, P.-T., Kroese, D. P., Mannor, S., and Rubinstein, R. Y. (2005). A tutorial on the cross-entropy method. *Annals of Operations Research*, 134(1):19–67.

Etienne, R. S., Haegeman, B., Stadler, T., Aze, T., Pearson, P. N., Purvis, A., and Phillimore, A. B. (2012). Diversity-dependence brings molecular phylogenies closer to agreement with the fossil record. *Proceedings of the Royal Society B*, 279(1732):1300–1309.

Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361.

Janzen, T., Höhna, S., and Etienne, R. S. (2015). Approximate Bayesian computation of diversification rates from molecular phylogenies: introducing a new efficient summary statistic, the nLTT. *Methods in Ecology and Evolution*, 6(5):566–575.

Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598):671–680.

Levine, R. A. and Casella, G. (2001). Implementations of the Monte Carlo EM algorithm. *Journal of Computational and Graphical Statistics*, 10(3):422–439.

Lewis, P. A. W. and Shedler, G. S. (1979). Simulation of nonhomogeneous Poisson processes by thinning. *Naval Research Logistics Quarterly*, 26(3):403–413.

Powell, M. J. D. (1994). A direct search optimization method that models the objective and constraint functions by linear interpolation. In Gomez, S. and Hennart, J.-P. (Eds.), *Advances in Optimization and Numerical Analysis*, pp. 51–67. Kluwer Academic Publishers.

Rubinstein, R. Y. (1999). The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190.

Wei, G. C. G. and Tanner, M. A. (1990). A Monte Carlo implementation of the EM algorithm and the poor man's data augmentation algorithms. *Journal of the American Statistical Association*, 85(411):699–704.

Table 1: Principal notation.

Symbol	Meaning
$\mathcal{T}$	Observed extant tree (phylo object or branching-time vector)
z	Unobserved extinct lineages
$\mathcal{T}^+ = (\mathcal{T}, z)$	Augmented tree: $\mathcal{T}$ together with extinct lineages z
$\mathbf{t} = (t_1 > \cdots > t_n > 0)$	Ordered branching times of the extant tree
$\boldsymbol{\theta} \in \mathbb{R}^8$	Full model parameter vector
$\lambda(s, t; \boldsymbol{\theta})$	Instantaneous speciation rate of lineage s at time t
$\mu(s, t; \boldsymbol{\theta})$	Instantaneous extinction rate of lineage s at time t
$N(t)$	Number of extant lineages at time t
$P(t)$	Total pendant branch-length at time t
$E(s, t)$	Evolutionary pendant time of lineage s
$p(\mathcal{T}, z \mid \boldsymbol{\theta})$	Complete-data likelihood
$p(\mathcal{T} \mid \boldsymbol{\theta})$	Marginal (observed-data) likelihood
$q(z \mid \mathcal{T}, \boldsymbol{\theta})$	Proposal density for extinct lineages
$\ell_i^{(f)}$	$\log p(\mathcal{T}, z_i \mid \boldsymbol{\theta})$
$\ell_i^{(q)}$	$\log q(z_i \mid \mathcal{T}, \boldsymbol{\theta})$
w_i	IS log-weight: $w_i = \ell_i^{(f)} - \ell_i^{(q)}$
S	IS sample size (augmented trees per particle)
M	CEM population size (particles per iteration)
$\widehat{f}(\boldsymbol{\theta})$	IS estimate of $\log p(\mathcal{T} \mid \boldsymbol{\theta})$
ESS	Effective sample size
ρ	Elite fraction retained per CEM iteration
T	Maximum number of CEM iterations
$\boldsymbol{\sigma}^{(k)}$	Per-parameter perturbation SD at CEM iteration k
α	Annealing decrement per iteration

Table 2: Model shortcuts and their free parameters.

String	Formula	Active covariates	Free parameters
`"cr"`	~ 1	none	$(\beta_0,\ \gamma_0)$
`"dd"`	$\sim N$	$N(t)$	$(\beta_0,\ \beta_N,\ \gamma_0,\ \gamma_N)$
`"pd"`	$\sim$ PD	$P(t)$	$(\beta_0,\ \beta_P,\ \gamma_0,\ \gamma_P)$
`"ep"`	$\sim$ EP	$E(s, t)$	$(\beta_0,\ \beta_E,\ \gamma_0,\ \gamma_E)$
—	$\sim N + \mathrm{PD}$	$N(t), P(t)$	$(\beta_0, \beta_N, \beta_P, \gamma_0, \gamma_N, \gamma_P)$
—	$\sim N + \mathrm{PD} + \mathrm{EP}$	all	all 8 parameters

Table 3: CEM control parameters (`estimate_rates_control("cem")`).

Parameter	Default	Description
`max_iter`	20	Hard iteration cap T
`num_points`	50	Population size M
`sample_size`	1	IS sample size S per particle per iteration
`shared_trees`	`FALSE`	Pool trees across particles (Mode 2)
`disc_prop`	0.5	Elite fraction ρ
`sd_vec`	`NULL`	Initial $\boldsymbol{\sigma}^{(0)}$; auto $= (u - \ell)/4$
`maxN`	10	Max augmentation attempts before rejection
`max_missing`	10^4	Max extinct lineages per augmentation
`max_lambda`	500	Thinning bound $\lambda_{\max}$
`tol`	10^{-4}	Plateau tolerance δ
`patience`	5	Consecutive plateau iterations P
`bias_correct`	`FALSE`	Moment-based bias correction (**??**)
`n_boot`	0	Bootstrap replicates for $\widehat{\mathrm{Var}}[\widehat{f}]$
`num_threads`	1	Parallel threads (Section **??**)

Table 4: MCEM control parameters (`estimate_rates_control("mcem")`).

Parameter	Default	Description
`sample_size`	200	IS sample size S per EM iteration
`tol`	0.1	Convergence tolerance ε
`burnin`	20	Iterations excluded from convergence check
`max_missing`	10^4	Max extinct lineages per augmentation
`max_lambda`	500	Thinning bound $\lambda_{\max}$
`num_threads`	1	Parallel threads (Section **??**)